

Q.1)

a) List and discuss advantages of Database Management System over file processing system.

- 1) Controlling Redundancy
- 2) Restricting Unauthorized Access.
- 3) Providing Persistent Storage for Program Objects.
- 4) Providing storage structures & Search Techniques for Efficient Query Processing
- 5) Providing Backup & Recovery.
- 6) Providing Multiple User Interface.
- 7) Representing Complex Relationships among Data.
- 8) Enforcing Integrity Constraints.
- 9) Permitting Inferencing and Actions Using Rules & Triggers.
- 10) Additional Implications of using the Database Approach.

i) → Redundancy in storing the same data multiple times leads to several problems.

i) There is the need to perform a single logical update such as entering data on a new student - multiple times: one for each file where student data is recorded. This leads to duplication of effort.

ii) Storage space is wasted when the same data is stored repeatedly, and this problem may be large serious for large databases.

iii) Files that represent the same data may become inconsistent. In the DB approach, the views of different users groups are integrated during the DB design. Ideally we should have a DB design that stores each logical data item - such in only one place in the DB. This is known as data normalization, and it ensures consistency & saves storage space.

2) → When multiple users share a large DB, it is likely that most users will not be authorized to access all information in DB.

3) → DB's can be used to provide persistent storage for programs & data structures. An object is said to be persistent, since it survives the termination of program execution & can later be directly retrieved by another program.

4) → The query processing & optimization module of the DBMS is responsible for choosing an efficient query execution plan for each query based on the existing storage structures.

5) A DBMS must provide facilities for recovering from HW & SW failure. The backup & recovery subsystem of the DBMS is responsible for recovery.

6) Many types of users with varying levels of technical knowledge use a DB, a DBMS should provide a variety of user interfaces. Ex: Query language for casual users, menu-driven interfaces for stand alone users.

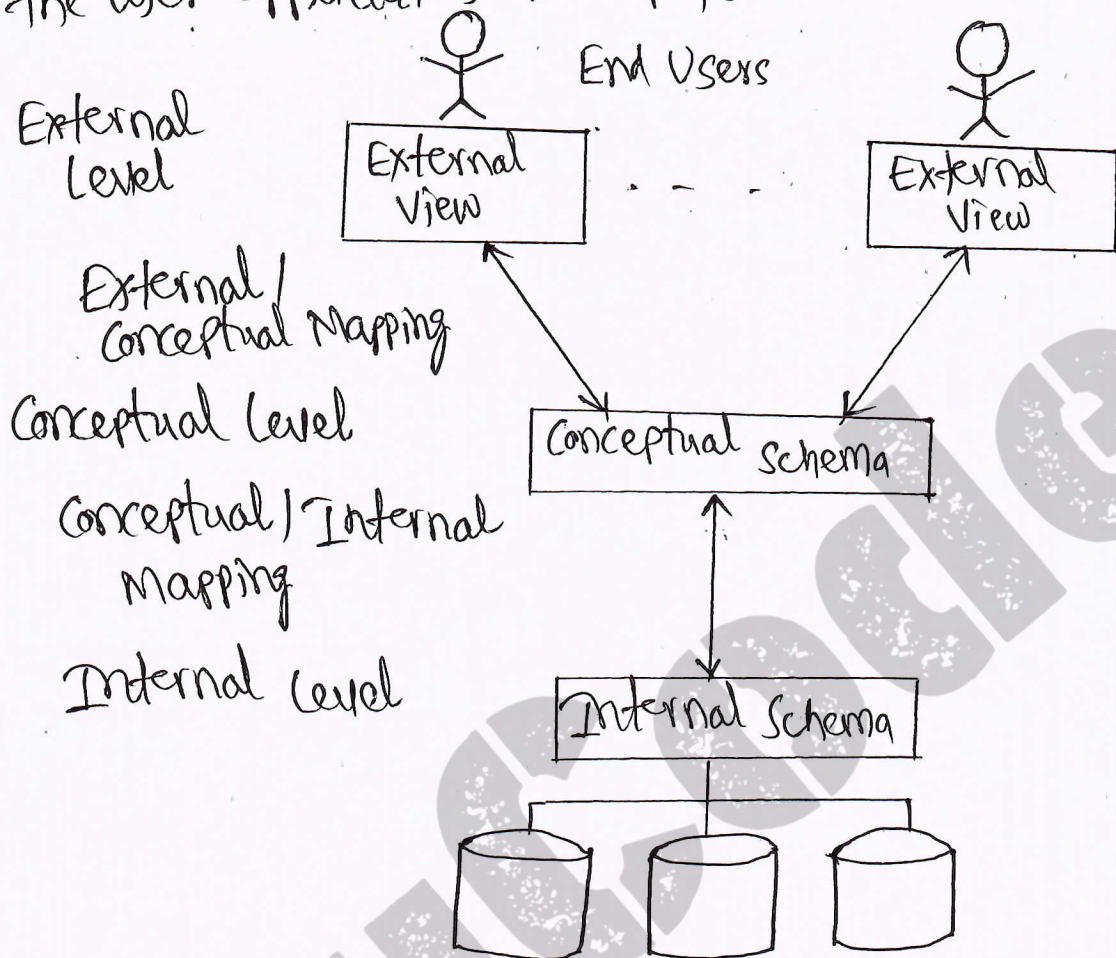
7) Representing Complex: A DBMS must have the capability to represent a variety of complex relationships among the data, to define new relationships as they arise & to retrieve and update related data easily & efficiently.

8) → Most DB applications have certain integrity constraints that must hold for the data. The simplest type of integrity constraint involves specifying a data type for each data item.

9) In a deductive DB system, one may specify declarative rules that allow to infer new data. Active DB systems go one step further by allowing "active rules" that can be used to initiate actions automatically.

b) Explain three schema architecture and reason for need of mapping among schema level.

→ The goal of the three-schema architecture is to separate the user applications from physical db.



In this architecture, Stored Database following three levels:

- 1) The internal level has an internal schema,
 - Describes the physical storage structure of the db.
 - Uses a physical data model & describes the complete details of data storage & access paths for the DB.
- 2) The conceptual level has a conceptual schema,
 - Describes the structure of the DB for a community users.
 - Hides the details of physical storage structures and concentrates on describing entities, relationships, user operations, & constraints.
 - Representational data model is used to describe the conceptual schema when a db system is implemented.
 - This implementation conceptual schema is often based on

only conceptual schema design model.

- 3) The External or view level includes a number of external schemas or user views,
- describes the part of the DB that a particular user group is interested in & hides the rest of the database from that user group.
 - As in the previous level, each external schema is typically implemented using a representational data model possibly based on an external schema design in a high-level data model.

The DBMS must transform a request specified on an external schema into a request against the conceptual schema, & then into a request on the internal schema for processing over the stored DB.

If the request is a DB retrieval, the data extracted from the stored DB must be reformatted to match the user's external level. The process of transforming requests & result between levels are called mappings.

c) Explain different types of attributes that occur in an E-R diagram model with example.

→ Types of attributes occur in the ER model.

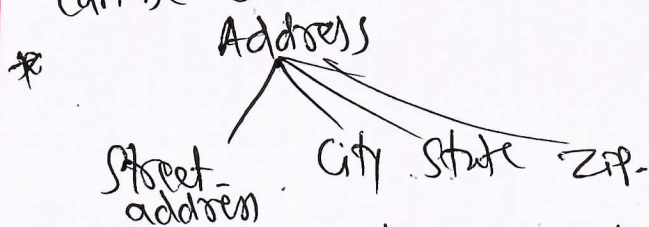
1) Simple Vs Composite.

2) Single valued Vs. multivalued

3) Stored Vs. Derived.

* Composite attributes can be divided into smaller subparts.

Eg:- The address attribute of the Employee entity can be divided into street address, City, State & Zip



* Attributes that are not divisible are called simple or atomic attributes.

Eg:- Age

* Single-valued :- Most attributes have a single value for a particular entity.

Eg: Age is a Single-valued attribute of a person.

* Multivalued :- An entity having multiple values for that attribute.

Eg: Color of a car = {black, red}.

* Stored & Derived Attribute:-

- Two attribute values are related. Eg: The age & Date attributes of a person.

- The value of age can be determined from the current date & the value of that person's Date.

- The age attribute is hence called a derived attribute.

- Date attribute is called a stored attribute.

2/4
Q1 Explain characteristics of the database approach.

→ 1) Self-Describing Nature of a DB System.

DB Approach :- A fundamental characteristic of the DB approach is that the DB system contains not only the DB itself but also a complete definition or description of the DB structure & constraints.

* This definition is stored in the DBMS catalog, which contains info. such as the structure of the file, the type & storage format of each data item, & various constraints on the data.

* The information stored in the catalog is called meta-data, it describes the structure of the primary DB.

Traditional File Processing:-

* In this data definition is typically part of the application programs themselves. Hence these programs are constrained to work with only one specific DB, whose structure is declared in the application programs.

2) Isolation between programs & data, & data Abstraction

* In traditional file processing, the structure of data files is embedded in the application programs, so any changes to the structure, if we want to add another piece of data require changing all pgrms that access that file.

- * By contrast, DBMS access programs do not require such changes in most cases. The structure of data files is stored in the DBMS catalog separately from the access programs. We call this property program-data independence.

3) Support of multiple Views of the Data:

→ A DB typically has many types of users, each of whom may require a different perspective or view of the DB.

- * A view may be a subset of the DB or it may contain virtual data i.e., derived from the DB files but is not explicitly stored.

- * A multiuser DBMS whose users have a variety of distinct applications must provide facilities for defining multiple views.

4) Sharing of Data & Multiuser Transaction Processing

- * A multiuser DBMS, as its name implies, must allow multiple users to access the DB at the same time.

- * This is essential if data for multiple applications is to be integrated & maintained in a single DB.

- * A fundamental role of multiuser DBMS software is to ensure that concurrent transactions operate correctly & efficiently.

- * A transaction is an executing program or process that includes one or more DB accesses, such as reading or updating of DB records.

b) Discuss the different types of User friendly interfaces.

→ 1) Menu-based Interfaces for web clients or Browsing. These interfaces present the user with lists of options (called menus) that lead the user through the formulation of a request.

2) Form-based interfaces displays a form to each user. Users can fill out all the form entries to insert new data or they can fill out only certain entries, in which case the DBMS will retrieve matching data for remaining entries.

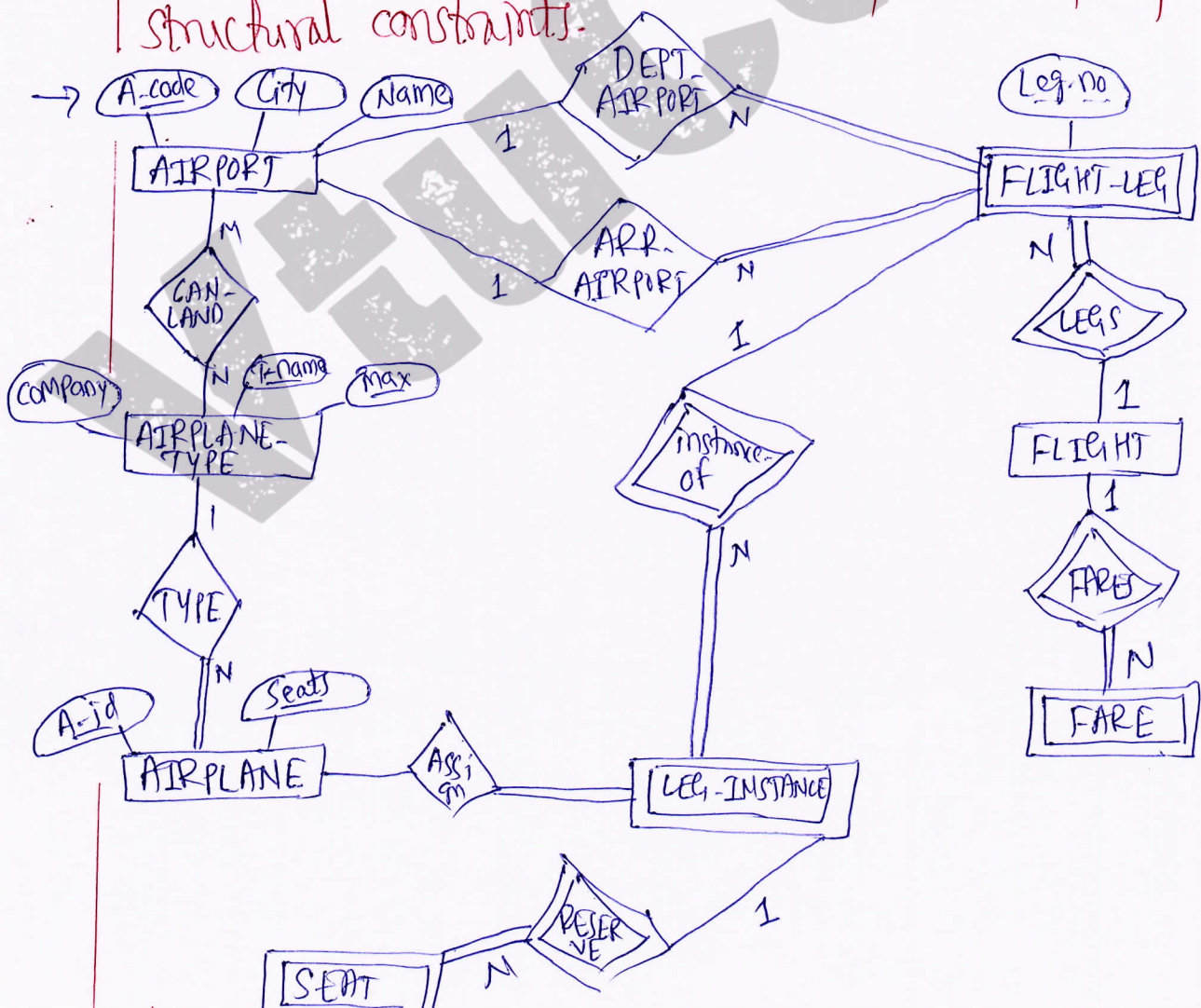
3) Graphical user Interfaces: displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram. GUIs utilize both menus & forms.

4) Natural language Interfaces: accepts requests written in English or some other language & attempt to understand them.

5) Speech input & output use of speech as an input query & speech as an answer to a question or result. The speech is detected using a library of predefined words & used to set up the parameters that are supplied to the queries.

6) Interfaces for parametric users such as bank tellers, often have a small set of operations that they must perform repeatedly.

c) Draw an ER diagram for an AIRLINES DB schema with atleast five entities. Also specify primary key and structural constraints.



Q. 2/4 What are the basic operations that can change the state of relations in the DB? Explain how the basic operations deal with constraint violations.

→ * Three basic operations that can change the states of relations in the DB.

i) Insert ii) Delete iii) Update

The Insert Operation: - The insert operation provides a list of attribute values for new tuple + that is to be inserted into a relation R.

* Insert can violate domain, key, entity and referential integrity constraints.

Operation:-

1) Insert < 'Cecilia', 'F', 'Kolonsky', NULL, '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4 > into Employee.

Result: It violates the entity integrity constraint, so it is rejected.

2) Insert < 'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4 > into EMP.

Result: - This insertion violates the key constraint, so it is rejected.

The Delete Operation: - The delete operation can violate only referential integrity will be violated.

Operation:-

Delete the Employee tuple with SSN = '999887777'.

Result: - This deletion is not acceptable, as it violates referential integrity.

The Update operation: - The update operation is used to change the values one or more attributes in a tuple of some relation. It is necessary to specify a condition on the attributes of the relation to select the tuple to be modified.

Update the Dno of the emp tuple with SSN = '99988' to 7.

→ Not acceptable, as it violates referential integrity.

- 07 Explain the terms Superkey, Candidate Key & Primary Key
- * Superkey is a single key or a group of multiple keys that can uniquely identify tuples in a table.
 - * A candidate key is a subset of Superkeys & is devoid of any unnecessary attributes that are not important for uniquely identifying tuples.
 - * The primary key uniquely identifies each record in a table. Primary keys must contain unique values, & cannot contain NULL values.

c) Given the following schema:

emp (fname, lname, ssn, bdate, address, gender, salary, SuperSSN, Dno)

dept (Dname, Dnumber, mgrSSN, mgrstartdate)

dept_loc (Dnumber, Dloc)

project (Pname, Pnumber, Ploc, Dnum)

works-on (ESSN, Pno, hours)

Dependent (ESSN, dependent-name, gender, bdate, relationship)

Give the relational algebra expression for the following.

i) Retrieve the name of the manager of each department.

$$R_1 \leftarrow \text{emp} \bowtie_{\text{ssn} = \text{mgr.ssn}} \text{dept}$$

$$R_2 \leftarrow \pi_{\text{fname}, \text{lname}}(R_1)$$

ii) For each project retrieve the project number, project name & no. of employees who worked on that project

$$R_1 \leftarrow \text{Project} \bowtie_{\text{Pnumber} = \text{Pno}} \text{works-on}$$

$$R_2 \leftarrow \pi_{\text{Pnumber}, \text{Pname}}(R_1), \text{count}(*)(R_1)$$

iii) Retrieve the names of employees who work on all the project controlled by department 5.

$$R_1 \leftarrow \rho_{\text{cno}}(\pi_{\text{Pnumber}}(\sigma_{\text{Dnum} = 5}(\text{PROJECT})))$$

$$R_2 \leftarrow \rho_{\text{ssn}, \text{Pno}}(\pi_{\text{ESSN}, \text{Pno}}(\text{WORKS-ON}))$$

$$R_3 \leftarrow \text{EMP} - R_1 \div R_2$$

$$R_4 \leftarrow \pi_{\text{name}, \text{fname}}(R_3 \star \text{EMP})$$

18) Retrieve The name of the employees who have no dependents.

→

$$R_1 \leftarrow \pi_{SSN}(EMP)$$

$$R_2(SSN) \leftarrow \pi_{SSN}(DEPENDENT)$$

$$R_3 \leftarrow R_1 - R_2$$

$$R_4 \leftarrow \pi_{Lname, Fname}(R_1 \star R_3)$$

19) Retrieve number of male & female employees working in the company.

$$\text{gender} \rightarrow \exists \text{ COUNT gender (EMP).}$$

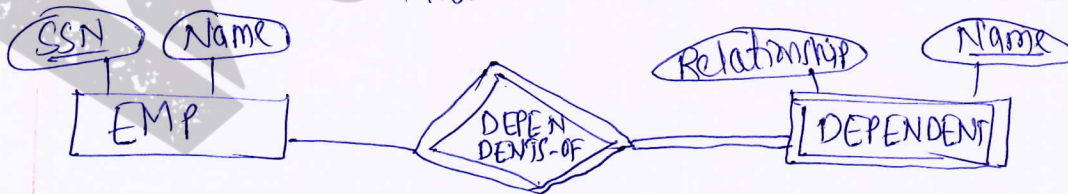
Q. 4/1 ay

Describe the steps of an algorithm for ER to Relational mapping with example.

→ Step 1: For each regular entity type E in the ER schema, create a relational R that includes all the simple attributes



Step 2: For each weak entity type W in the ER schema with owner entity type E, create a relation R, & include all simple attributes of W as attributes. In addition, include as foreign key attributes of R the primary key attribute of the relation that correspond to the owner entity type.



DEPENDENT

SSN | Relationship | Name

Step 3: For each binary 1:1 relationship type R identify the relation (N) relation S. The primary key of T as a FK of S. Simple attributes of R map to attributes of S.

EMP. DEPARTMENT

Mgrs-SSN | Start-Date

Step 4: For each regular binary 1:N relationship type R, identify the relation (N) relation S. The primary key of T as a FK of S. Simple attributes of R map to attributes of S.

Emp
[Superssn]

Step 5: For each binary M:N relationship type R, create a relation S. Include the PK's of participant relations as FK in S. Their combination will be the PK for S. Simple attributes of R become attributes of S.

works FOR
[ESSN | Dnumber]

Step 6: For each multivalued attribute A, create a new relation. This relation will include an attribute corresponding to A, plus the PK K of the parent relation as a FK in R. The PK of R is the combination of A & K.

DEPT-LOC.

[Dloc | Dnumber]

Step 7: For each n-ary relationship type R, where $n \geq 2$, create a new relation S to represent R.

by write command that is used for table creation. Explain how constraints are specified in SQL during table creation, with suitable example.

→ The create table command is used to specify a new relation by giving its name & specifying its attributes and initial constraints.

is NOT NULL : It is specified if NULL is not permitted for particular attribute.

Create table EMP (Fname^{varchar(20)} NOT NULL, Lname varchar(20))

ii) DEFAULT :- It is possible to define a default value for an attribute by appending the clause Default to an attribute definition.

Create table DEPT (.....
mgr_ssn (char(9) NOT NULL DEFAULT 'SSSSSSSS')
.....)

17) Given the following schema:

Emp(Fname, Lname, SSN, bdate, address, gender, salary,
SupersSN, dno)

dept(dname, dnumber, mgrssn, mgrstartdate)

dept_loc(dnumber, dloc)

project(Pname, Pnumber, Ploc, dnum)

works-on(ESSN, Pro, hours)

dependent(ESSN, dependant-name, gender, bdate, relationship)

Give the relational algebra expression for the following

i) Retrieve the name & address of all employees who work for 'Sports' department

$$R_1 \leftarrow \sigma_{\text{dname} = \text{'Sports'}}(\text{Dept})$$
$$R_2 \leftarrow R_1 \bowtie_{\text{dnumber} = \text{dno}} \text{Emp}$$
$$R_3 \leftarrow \pi_{\text{Fname}, \text{Lname}, \text{Address}}(R_2)$$

ii) Retrieve each dept number, number of employees & their average salary.

$$\text{dno} \int (\text{COUNT}(SSN), \text{AVERAGE salary}(\text{EMP}))$$

iii) List the project number, controlling department numbers & department manager's last name, address & birthdate.

$$R_1 \leftarrow \text{PROJECT} \bowtie_{\text{dnum} = \text{dnumber}} \text{DEPT}$$
$$R_2 \leftarrow R_1 \bowtie_{\text{mgr-SSN} = \text{SSN}} \text{EMPLOYEE}$$
$$R_3 \leftarrow \pi_{\text{Pnumber}, \text{Dnum}, \text{Lname}, \text{Address}, \text{Bdate}}(R_2)$$

iv) Retrieve the name of the employees with 2 or more dependents.

$$T_1(\text{SSN}, \text{No. of - depen} \leftarrow \text{dents}) \leftarrow \text{ESSN} \int \text{count dependant-name}(\text{DEPENDENT})$$
$$T_2 \leftarrow \sigma_{\text{No of - dependents} > 2}(T_1)$$
$$T_3 \leftarrow \pi_{\text{Lname}, \text{Fname}}(T_2 \bowtie \text{EMP})$$

Q 5/97 Define database stored procedure. Explain creating & calling stored procedure with example.

- * A stored procedure is a pgm. that is executed through a single SQL statement that can be locally executed & compiled within the process space of the database server.
- * The results can be packaged into one big result & returned to the application, or the application logic can be performed directly at the server, without having to transmit the results to the client.

Creating a Simple Stored Procedure

```
CREATE PROCEDURE ShowNumberoforders  
SELECT C.cid, C.cname, COUNT(*)  
FROM CUSTOMERS C, ORDERS O  
where C.cid = O.cid  
Group by C.cid, C.cname
```

Calling Stored Procedures:

- * Stored procedures can be called in interactive SQL with the CALL statement.

```
CREATE PROCEDURE RankCustomers (IN number INTEGER
```

Language Java

External Name 'file:///c:/stored procedures/rank.jav

```
CALL storedprocedureName (argument 1, argument 2, ...,  
argument N);
```

- * Calling Stored Procedures from JDBC.

Callable Statement cstmt = con. prepareCall ("{call
showNumberoforders ?}");

ResultSet rs = cstmt.executeQuery();
while (rs.next())

by what is SQLJ and how it is different from JDBC

-
- * SQLJ was developed to complement the dynamic way of creating queries in JDBC with static model.
 - * Unlike JDBC, having semi-static SQL queries allows the compiler to perform SQL syntax checks, storing type checks, of the computability of the host variables with the respective SQL attributes.
 - * In SQLJ, variables in the host language allows are bound statically to the same arguments, whereas in JDBC, we need separate statements to bind each variable to an argument & to retrieve the result.
 - * SQLJ pgms require fewer lines of code than JDBC pgms. They are shorter & hence easier to debug.
 - * SQLJ provides strong type checking of query results & other return parameters, while JDBC values are passed to & from SQL without having been checked at compile time.
 - * SQLJ fills a complementary role for static SQL, while JDBC provides a complete dynamic SQL interface from Java to relational DBs.
 - * SQLJ is used to write a pgm when you want to write an application that you can deploy to another DB, while, JDBC is used when you do not want to have a SQLJ layer during deployment.
 - * SQLJ is used when you want to be able to check your pgm for errors at translation time rather than at run time while JDBC is used when your pgm uses dynamic SQL.

c) Consider the following schema:

- Sailors (Sid, Sname, rating, age)
- Boats (bid, bname, color)
- Reserves (Sid, bid, day)

Note: anomalies in SQL

ii) Find the ages of sailors whose name begins & ends with A & has atleast three characters.

```
select age
from Sailors
where Sname like 'A_ _%A';
```

iii) Find the age of the youngest sailor who is eligible to vote for each rating level with atleast two such sailors.

```
select S.rating, MIN(S.age)
from Sailors S
where S.age > 18
group by S.rating
having 1 < (select count(*)
from Sailors S2
where S.rating = S2.rating);
```

iii) Find the names of sailors who have not reserved a red boat

```
select S.name
from Sailors S
where S.sid not in (select sid
from Sailors S, reserves R, boats B
where S.sid = R.sid & R.bid = B.bid & B.colour = 'red');
```

iv) Compute increments for the rating of persons who have sailed two different boats on the same day.

```
select rating+1
from sailor
where sid in (select sid
from Reserves R
group by day, sid
having count(*) = 2)
```

Q6)

a) what is CGI ? why was CGI introduced ? what are the disadvantages of an architecture using CGI script ?

→ * The CGI connects HTML forms with application pgms.

* It is protocol that defines how arguments from forms are passed to come at the server side

* Prgms that communicate with web server via CGI are often called CGI scripts.

* CGI was introduced to transmit arguments from HTML forms to application prgms running at the middle tier.

* Interaction with standalone application was costly.

The disadvantages of CGI are

1) Each page request results in the creation of a new process.

2) Data can not be easily cached betn page loads.

3) It uses a lot of processing time.

4) No resource sharing betn application prgms.

b) What is Dynamic SQL & how is it different from embedded SQL? Explain.

→ * We may not be able to execute, even though there is some algorithm by which the application can construct the necessary SQL statements once a users command is issued. SQL provides some facilities to deal with such situations. These are referred to as Dynamic SQL.

* It is a programming technique that enables you to build SQL statements dynamically at runtime.

* The preparation of a Dynamic SQL command occurs at runtime & is runtime overhead whereas Embedded SQL commands can be prepared once at compile time & re-executed as often desired.

* Dynamic SQL is less efficient but Embedded SQL is more efficient.

* Dynamic SQL is more flexible than Embedded SQL.

* In Dynamic SQL, EXECUTE, PREPARE statements are used. In embedded SQL, EXECUTE, PREPARE statements are not used.

c) Consider the following schema:

Sailors (Sid, Sname, rating, age)

Boats (bid, bname, color)

Reserves (Sid, bid, day)

write queries in SQL.

i) Find the names of sailors who have reserved atleast one boat.

```
SELECT DISTINCT S.sname
FROM Sailors S, Reserves R
WHERE S.sid = R.sid.
```

ii) Find sailors whose rating is better than some sailor called 'Jennifer'.

```
SELECT S.sname, S.rating
```

```
FROM Sailors
```

```
WHERE S.rating > ALL (SELECT S1.rating
FROM Sailors S1
WHERE S1.sname = 'Jennifer')
```

iii) Find the average age of sailor for each rating level that atleast two sailors.

```
SELECT S.rating, AVG(S.age)
```

```
FROM Sailors S
```

```
GROUP BY S.rating
```

```
HAVING COUNT(*) > 1
```

iv) Find the name & age of the oldest sailor.

```
SELECT S.sname, S.age
```

```
FROM Sailors S
```

```
WHERE S.age = (SELECT MAX(S2.age)
FROM Sailors S2).
```

Q. 7)

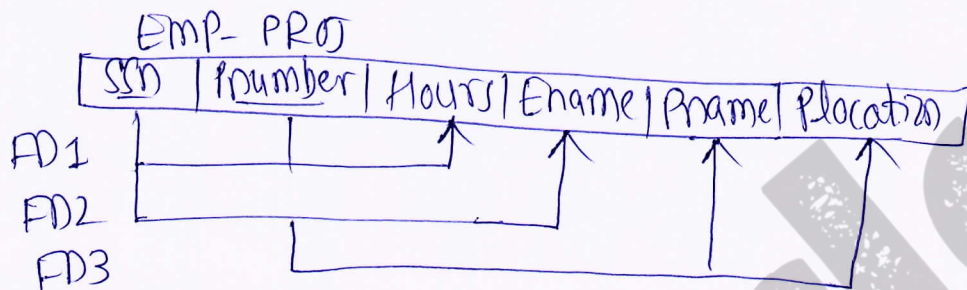
a) which normal form is based on 6 transitive FD's & full functional dependency & Explain the same with ex.

→ 2NF is based on 6 transitive FD's & full functional dependency.

Second Normal Form

* A FD $X \rightarrow Y$ is a full FD if removal of any attribute from X means that the dependency does not hold anymore.

* A FD $X \rightarrow Y$ is a partial dependency if some attribute $A \in X$ can be removed from X & the dependency still holds.



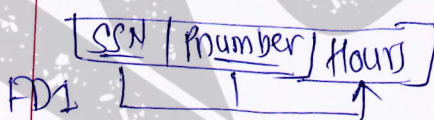
* In the above figure, $\{SSN, Pnumber\} \rightarrow \{Hours\}$ is a full dependency.

* $SSN \rightarrow Ename$ & $Pnumber \rightarrow \{Pname, Location\}$ is a partial dependency.

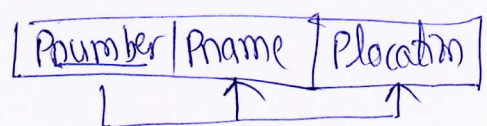
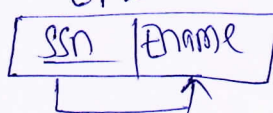
Defn: - A relation schema R is in 2NF if every nonprime attribute A in R is a fully functionally dependent on the PK of R .

↓ 2NF Normalization.

EP1



EP2



by A relation R satisfies the following FD's:

$A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H$. Find the cover for this set of FD's.

→

i) $A \rightarrow C$

ii) $AC \rightarrow D$

iii) $E \rightarrow A$

iv) $E \rightarrow D$

v) $E \rightarrow H$

Decomposition.

Remove $E \rightarrow D$ Redundant

$$AC \rightarrow D$$

$$A^+ = \{A, C\}$$

$\therefore C$ is extraneous & is removed.

Minimal cover $\Sigma := \{A \rightarrow CD, E \rightarrow AH\}$.

c) Consider the universal relation: $R = \{A, B, C, D, E, F, G, H, I, J\}$
 & the set of FD's, $F = \{AB \rightarrow C, A \rightarrow DE, B \rightarrow F, F \rightarrow GH, D \rightarrow IJ\}$.
 Determine whether each decomposition has loss less join property
 w.r.t F . $D_1 = \{R_1, R_2, R_3\}$; $R_1 = \{A, B, C, D, E\}$;
 $R_2 = \{B, F, G, H\}$; $R_3 = \{D, I, J\}$.

$$\rightarrow F = \{AB \rightarrow C, A \rightarrow DE, B \rightarrow F, F \rightarrow GH, D \rightarrow IJ\}$$

$$D_1 = \{R_1, R_2, R_3\}$$

$$R_1 = \{A, B, C, D, E\}$$

$$R_2 = \{B, F, G, H\}$$

$$R_3 = \{D, I, J\}$$

$$R_1 \cup R_2 \cup R_3 = R \text{ (universal relation).}$$

$$R_1 \cap R_2 = \emptyset$$

$$R_2 \cap R_3 = \emptyset$$

$$R_1 \cap R_3 = \emptyset$$

$\therefore$ Hence each decomposition has loss less join property with respect to F .

8) Write an algorithm to check whether decomposed relations are in 3NF with dependency preservation & non-additive join property. Consider universal relation $R = (U, C, L, A)$ & the set of FD's. $F = \{P \rightarrow LCA, LC \rightarrow AP \rightarrow A \rightarrow C\}$. Decompose the relation R into 3NF with dependency preservation & non-additive join property.

Algorithm for Testing for Non-additive Join property

Input: A universal relation R , a decomposition $D = \{R_1, R_2, \dots, R_i\}$ of R , & set F of FD's.

1) Create an initial matrix S with one row i for each relation R_i in D & one column j for each attribute A_j in R .

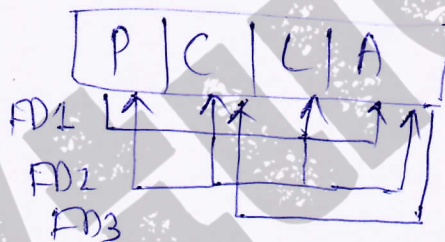
2) Set $S(i, j) := b_{ij}$ for all matrix entries.

3) for each row i representing relation schema R_i
& for each column j representing attribute A_j
& for (relation R_i includes attribute A_j) then set
 $S(i, j) := a_{ij} \{; \}$

4) Repeat the loop until a complete loop execution results in no changes to S .

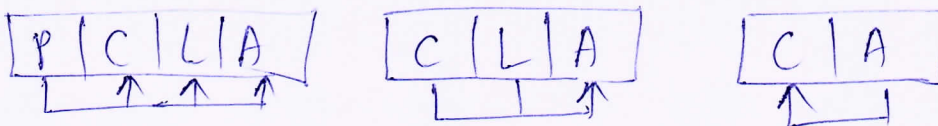
5) If a row is made up of entirely of a symbols, then the decomposition had the nonadditive join property; otherwise it does not.

$R = (U, C, L, A)$



From the diagram it is clearly visible that it is not 3NF as it has transitive dependency.

Decompose table into 3 subtables as



Now it is 3NF.

b) Define Normal form. Explain 1NF, 2NF, 3NF with suitable example for each.

→ The normal form of a relation refers to the highest normal form condition that it meets, & hence indicates the degree

which it may have been normalised.

1NF:- 1NF disallows repeating values within relations or relations as attributes values within tuples.

* The only attribute values permitted by 1NF are single atomic values.

DEPT

<u>Dname</u>	<u>Dnumber</u>	<u>Dmgr-SSN</u>	<u>Dlocations</u>
--------------	----------------	-----------------	-------------------

↓ 1NF

DEPT

<u>Dname</u>	<u>Dnumber</u>	<u>Dmgr-SSN</u>	<u>Dlocations</u>
Research	5	111	{B, C, D}
CS	4	222	{C}
EC	1	444	{B}

↓ 1NF

<u>Dname</u>	<u>Dnumber</u>	<u>Dmgr-SSN</u>	<u>Dlocations</u>
Research	5	111	B
Research	5	111	C
Research	5	111	D
CS	4	222	C
EC	1	444	B

2NF:- 2NF is based on the concept of full functional dependency.

* A FD $x \rightarrow y$ is a full functional dependency if removal of any attribute A from x means that the dependency does not hold any more, i.e., for any attribute $A \in x$, $(x - (A))$ does not functionally determine y.

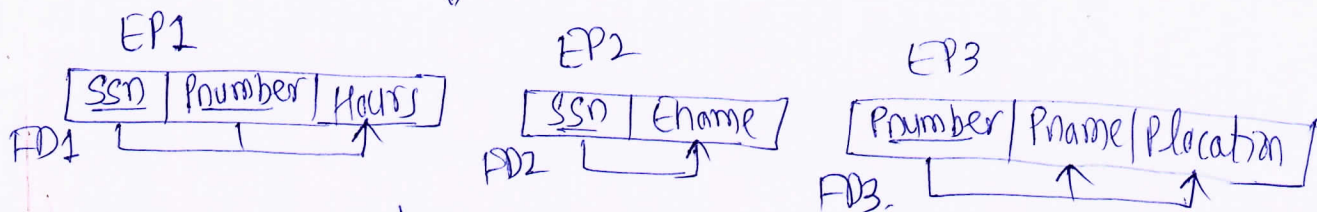
* A relation schema R is in 2NF if every nonprime attribute A in R is fully functionally dependent on the PK of R.

EMP-PROJ

<u>SSN</u>	<u>Pnumber</u>	<u>Hours</u>	<u>Ename</u>	<u>Pname</u>	<u>Plcation</u>
------------	----------------	--------------	--------------	--------------	-----------------

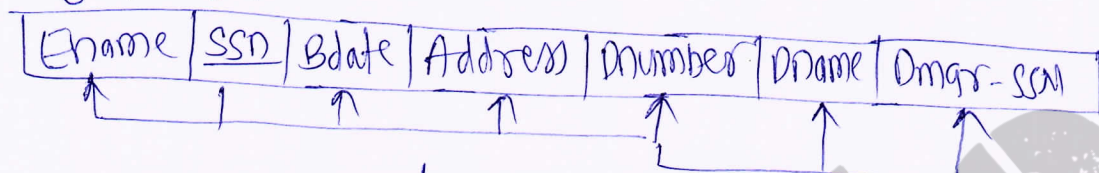
FD1: SSN → Hours
FD2: SSN → Ename
FD3: SSN → Pname

↓ 2NF

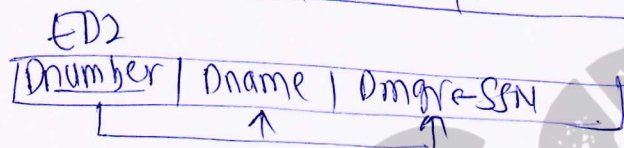
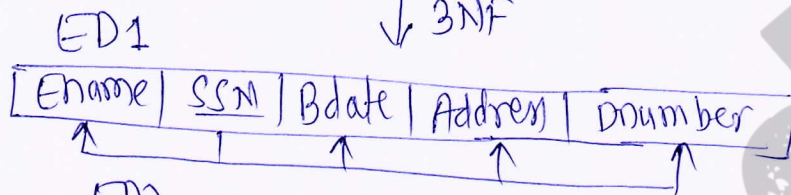


* 3NF :- A relation Schema R is in 3NF if it is 2NF & no non-prime attribute A in R is transitively dependent on the PK

EMP-DEPT



↓ 3NF



Q) Consider two set of FD's $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ & $G = \{A \rightarrow CD, E \rightarrow AH\}$. Are they equivalent?

→ F: $A \rightarrow C$
 $AC \rightarrow D$
 $E \rightarrow AD$
 $E \rightarrow H$

G: $A \rightarrow \cancel{A}CD$
 $E \rightarrow AH$

$A^+ : ACD$
 $(AC)^+ : ACD$
 $(E)^+ : ACDHE$

$A^+ : ACD$
 $E^+ : ACDEH$

$F \subseteq G$

$G \subseteq F$

$F = G$

Q.9)

What are the anomalies occur due to interleaved execution? Explain them with example.

→ The Lost update Problem:- Occurs when two transactions that access the same DB items have their operations interleaved in a way that makes the value of some DB item incorrect.

T ₁	T ₂
$r(x)$ $x = x - N$	$r(x)$ $x = x + M$
$w(x)$ $r(y)$	$w(x)$
$y = y + N$ $w(y)$	

2) The dirty read problem; - occurs when one transaction updates a db item & the transaction fails for some reason. * Meanwhile the updated item is accessed by another transaction before it is changed back to its original value.

T ₁	T ₂
$r(x)$ $x = x - N$ $w(x)$	$r(x)$ $x = x + M$ $w(x)$
$r(y)$	

3) The incorrect summary problem: If one transaction is calculating an aggregate summary function on a number of db items while other transactions are updating some these items.

T ₁	T ₂
	$sum = 0$ $r(A)$ $sum = sum + A$ $\vdots$
$r(x)$ $x = x - N$ $w(x)$	
	$r(x)$ $sum = sum + x$ $r(y)$ $sum = sum + y$
$r(y)$ $y = y + N$	

4) The Unrepeatable Read Problem: Transaction T reads the same item twice & gets different values on each read, since the item was modified by another transaction T' betn the two reads.

b) Explain different types of locks used in concurrency control.

→ 1) Binary locks: → A binary lock can have two states: locked & unlocked.

* If the value of the lock on x is 1, item x can not be accessed by a db operation that requests the item.

* If the value of the lock on x is 0, the item can be accessed when requested, the lock value is changed to 1.

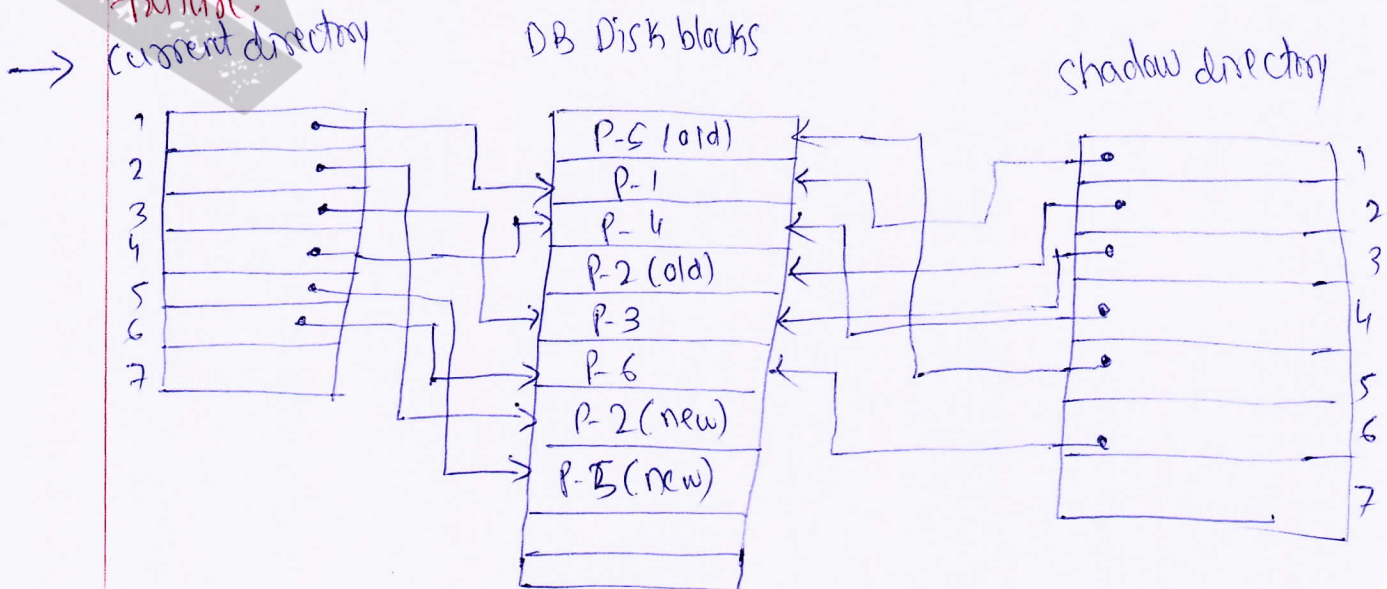
2) Shared/Exclusive locks:-

* In this scheme there are 3 locking operations: read-lock(x), write-lock(x), and unlock(x).

* A read-locked item is called shared-locked because other transactions are allowed to read the item.

* A write-locked item is called exclusive-locked because a single transaction exclusively holds the lock on the item.

c) Explain how shadow paging helps to recover from transaction failure.



* This recovery scheme does not require the use of a log in a single-user environment.

* Shadow paging considers the db to be made up of a no. of fixed size disk pages - say, n - for recovery purposes.

* A directory with n entries is constructed, where the i th entry points to the i th db page on disk.

* The directory is kept in main memory if it is not too large & all references - reads or writes - to database pages on disk go through it.

* When a transaction begins, executing, the current directory - whose entries point to the most recent or current db pages on disk - is copied into a shadow directory.

* The shadow directory is then saved on disk while the current directory is used by the transaction.

* During the transaction execution, the shadow directory is never modified.

* The old version is referenced by the shadow directory & the new version by the current directory.

Q. 10) Explain ACID property of transaction and system log.

→ a) Atomicity:- A transaction is an atomic unit of processing & it is either performed entirely or not at all.

b) Consistency Preservation:- A transaction should be consistency preserving, that is it must take the db from one consistent state to another.

c) Isolation:- A transaction should appear as though it is being executed in isolation from other transactions.

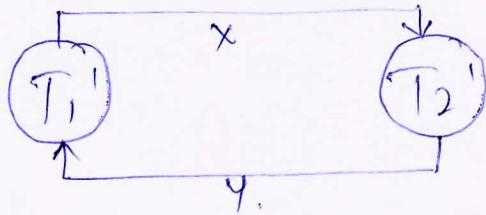
d) Durability:- If a transaction changes the db and is committed, the changes must never be lost because of any failure.

→ The system log:- It keeps track of all transaction operations that affect the values of db items.

* This info may be needed to permit recovery from transaction failures.

b) When deadlock and starvation problem occurs? Explain how these problems can be resolved.

→ * Deadlock occurs when each transaction T in a set of two or more transactions is waiting for some item that is locked by some other transaction T' in the set.



* Starvation may occur when we use a transaction can not proceed for an indefinite period of time while other transactions in the system continue normally.

* Deadlock can be resolved by.

1) Deadlock prevention protocols.

- a) Conservative two-phase locking
- b) ordering of all items.
- c) wait-die
- d) wound-wait
- e) No waiting algorithm
- f) cautious waiting

2) Deadlock Detection.

- a) wait-for graph
- b) Timeouts.

* Starvation problem can be resolved by:

- a) A Fair waiting queue.

c) Explain ARIES recovery algorithm with example.

→ * ARIES uses a steal/no-force approach for writing. It is based on three concepts.

- 1) write-ahead logging
- 2) Repeating history during redo.
- 3) logging changes during undo

* The ARIES recovery procedure consists of 3 main steps.

1) Analysis.

2) REDO

3) UNDO.

* The Analysis step:-

a) Identifies the dirty pages in the buffer & the set of transactions active at the time of the crash.

b) The appropriate point in the log where the REDO operation should start is also determined.

* The REDO phase:-

- reapplies updates from the log to the db.
- Certain info in the ARIES log will provide the start point for REDO, from which REDO operations are applied until the end of log is reached.

* The UNDO phase:-

- The log is scanned backward & the operations of transactions that were active at the time of the crash are undone in reverse order.